

1. Khalid - Project Lead, Backend Architecture, Simulation Engine

Main Responsibilities

Khalid handled the main backend architecture and coordinated the project flow.

His areas:

- Overall project planning.
- Backend folder structure.
- Flask app setup.
- Simulation engine design.
- Scenario manager.
- WebSocket integration.
- Final frontend-backend integration.

Modules Owned

- `backend/app/__init__.py`
- `backend/app/routes/simulation.py`
- `backend/app/simulation_engine/`
- `backend/app/realtime/events.py`
- `backend/app/extensions.py`

Work Details

Khalid designed the system so every attack scenario returns the same type of data:

- Scenario details.
- Attack success or failure.
- Metrics.
- Packets.
- Alerts.
- Hacker logs.
- Defender logs.
- Active defenses.
- Realtime event stream.

This made the frontend reusable because the dashboard did not need a different layout for every attack.

How He Should Explain His Work

I worked on the backend architecture and simulation engine. I designed a common response structure for all attack scenarios so the frontend could show every scenario in the same dashboard. I also integrated Flask-SocketIO so the backend can stream live events like packet movement, attack logs, and defense alerts.

Key Points To Mention

- Backend is modular.
- Every API response follows the same JSON pattern.
- All four scenarios are backend-generated.
- WebSockets make the dashboard real-time.
- REST APIs still exist as backup and for initial state.

Khalid Presentation Guide

What Khalid Should Say

My role was project leadership, backend architecture, simulation engine, and real-time integration. I structured the backend so each scenario follows one common format. This format includes metrics, packets, alerts, attack logs, defense logs, active defenses, and an event stream. Because of this, the frontend can display all scenarios using the same dashboard components.

Work Area Explanation

Khalid worked on the backend foundation.

How it was implemented:

1. Flask app factory was created.
2. Routes were separated into modules.
3. Simulation engine folder was created.
4. Each attack scenario was implemented as a separate file.
5. A scenario service chooses the correct scenario builder.
6. Event stream builder converts scenario data into live WebSocket events.
7. Flask-SocketIO sends those events to the frontend.

Modules To Mention

- `simulation_service.py`
- `scenario_catalog.py`
- `bank_mitm_scenario.py`

- password_bruteforce_scenario.py
- packet_sniffing_scenario.py
- replay_attack_scenario.py
- event_stream.py
- events.py

Viva Questions For Khalid

Q: Why did you make a simulation engine instead of writing everything directly in frontend?

A: Because the backend should control attack logic. This makes scenarios reusable, testable, and closer to real system architecture.

Q: What does the simulation engine return?

A: It returns scenario information, metrics, packets, alerts, attack logs, defense logs, defense status, and event stream.

Q: Why use WebSockets?

A: WebSockets allow the backend to push live updates to the frontend without waiting for repeated HTTP requests.

Q: What is the benefit of modular backend design?

A: Each module has one responsibility, so the system is easier to maintain and expand.

Q: Which scenario is most important?

A: Bank MITM is the clearest cryptography demonstration because it shows plaintext interception and encrypted protection.